「AI 时代的思考」

从认知革命到商业落地

Kingson Wu

2026 年 3 月

本册写于 2025 – 2026 年，部分观点可能已有变化、存在错误，或视角狭隘、覆盖不全。
内容主要针对 Transformer 架构下的生成式 AI，请结合自己的思考和当下的现实情况批判性阅读。
建议将本册作为思考的参考，导入任意 AI 软件进行问答与批判，
最终形成你自己的观点。

目　录

第一部分：理解 AI

AI工业革命下的若干思考
深度学习为什么能学习并做对
人类与模型的共同特征：直觉先行，解释在后

第二部分：人机关系

关于AI替代人类工作的问题
技术进步与分配失衡：AI时代的结构性矛盾
从Prompt到碳基Agentic RAG：AI时代的人类认知外化
AI与学习型人际关系的变化

第三部分：创造与边界

AI能否创造真正的新事物
AI生成内容时代：原创性、作者身份与创作价值的再定义
生成式AI的概率本质与创新边界：从一个具体任务说起

第四部分：架构与商业

从自建Agent到Agent Runtime + Skill：AI系统架构的演进与收敛
将执行路径固化为资产：一种面向长期运行的AI架构
大模型时代：中小公司如何寻找AI竞争空间
从Skill化到控制权：AI系统中的能力分层与资产保护

前言

这不是一份技术手册，也不是一份系统性教材。

过去两年，AI 领域发生了前所未有的变化。模型能力在某些维度上已经超出了大多数人的预期，但与此同时，关于 AI 能做什么、不能做什么、将会如何影响我们的工作和生活——这些基本问题，答案依然模糊。恐惧与期待并存，焦虑与兴奋交织。大多数讨论要么过于技术，要么过于抽象，真正面向「有技术背景但不需要实现细节」的产品决策者视角的思考，并不多见。

这本小册子正是这个方向的尝试。作者从自身实践和观察出发，围绕 AI 的本质、人机关系、创造边界和商业落地四个层面，整理了过去两年对 AI 的一些系统性思考。这些思考未必完整，也未必都对，但它们是真实的。

全册分为四个部分：

第一部分「理解 AI」，从工业革命的历史框架出发，讨论 AI 的本质是「规律发现系统」而非类人智能，探讨深度学习为什么能工作，以及人类认知模式与大模型的深层相似性。

第二部分「人机关系」，直面 AI 对就业市场的冲击，以及由此引发的结构性分配矛盾；讨论人如何在 AI 时代重新定位自己的认知角色，以及知识型人际关系正在发生的变化。

第三部分「创造与边界」，系统分析 AI 创造力的真实边界——它擅长组合优化，但难以实现概念跃迁；进一步讨论 AI 生成内容时代，原创性和作者身份如何被重新定义。

第四部分「架构与商业」，从系统架构演进的角度，理解 Agent Runtime 与 Skill 体系的分化逻辑，讨论如何将执行路径固化为长期资产，以及在 Skill 化趋势下如何保护核心能力；最后回到商业视角，探讨中小公司在 AI 生态中的竞争空间。

四部分构成了一个完整的认知链条：从理解 AI 是什么，到认识它如何影响我们，到厘清它的能力边界，再到思考如何用它构建可持续的系统。阅读顺序即是这个逻辑链条的展开顺序。

希望这些思考对你有用，哪怕只是让你在某个问题上多了一个角度。

AI工业革命下的若干思考

>

AI对就业的双重影响

- AI技术对就业市场带来了创造效应和破坏效应：
  - 创造效应：AI的普及将催生了许多潜在的新兴职业。这些新兴岗位对熟练掌握AI工具的专业人才需求旺盛。
  - 破坏效应：AI替代了许多重复性、规则性较强的岗位。某些中等技能的岗位如流水线操作、传统文秘工作可能面临消失的风险。

历史视角：被替代的人何去何从

- 随着AI技术的发展，未来许多人的工作将不可避免地被替代，这其中包括一部分程序员，但相较而言，更多非程序员的工作可能会首当其冲。然而，那些更擅长使用并深刻理解AI工具的人将更有可能保住自己的职位。
- 回顾工业革命，技术进步曾一度导致大批劳动者失业，许多人因无法适应新变化而成为时代的“牺牲品”，收入锐减。
- 短期内，某些群体的失业可能难以避免，但历史也表明，技术变革往往伴随着新机会的诞生。
- 所以，个人需要做好准备：
  - 提升自身技能，特别是AI工具的使用和相关领域的知识储备。
  - 即便不能马上参与高端AI开发，也可以从简单的AI相关工作入手，逐步积累经验。
  - 有一定资产的群体，应学会投资，通过多样化方式应对收入变化。

正面效应

- 催生新型岗位：AI技术完善后，将需要大量擅长操作和优化AI工具的人才。
- 解放低级劳动：AI让人类摆脱低级脑力劳动，从而获得更高工资和更有意义的工作体验。
- 效率提升与“去内卷”：生产效率的提升可能减少“内卷”现象，并通过政策调整减轻劳动者的工作压力。
- 关于工作和财富再分配问题：如果AI真的让生产效率大幅提升，我们可以期待未来的福利制度更加完善，比如减少工作时间、增加带薪假期，甚至实现“无工作”状态也未必是幻想。

社会与政策层面

- 加强教育培训：推动技能提升和再教育计划，使劳动者适应AI时代的工作需求。
- 优化产业结构：加速产业转型升级，为社会创造更多高附加值就业机会。
- 完善社会保障：建立健全失业保障制度，帮助因AI技术变革失业的劳动者渡过难关。

- 政策引导：通过政策规范AI技术的合理应用，避免过度自动化对就业的过度冲击。

总结

- AI技术的快速发展是一把双刃剑，它既对传统就业模式提出了挑战，也为社会提供了提高效率、优化劳动分工的契机。从社会视角看，技术进步不可逆，但我们可以通过政策调整和个人适应来尽量减小负面影响。面对不可避免的变革，积极调整心态、提升能力，是每个人应对未来的重要策略。同时，历史告诉我们，每一次技术浪潮都会带来新的机遇，只要抓住变化的窗口期，未来依然充满希望。

深度学习为什么能学习并做对

我的理解

- 目前无法解释为什么深度学习可以给出答案，只能大概解释为什么它能学习
- 这就相当于：我不能解释为什么这个人是通过什么做对这道题的，我只能解释他有正常的学习能力以及合适的学习资料，所以他自己能学习并做对题目
- 所以深度学习是怎么给出答案的，后续或许应该由它本身来告诉我们。
- 依据不同人的认知水平，来告诉我们它通过学习所发现的规律。同时人类通过它能快速学习发现事物规律的能力，来进一步帮助提升人类的认知。

AI对我理解的看法

>

- 你理解的方向是对的，可以稍微精炼和调整一下：
  - 深度学习的一个核心问题就是我们无法完全解释它是如何给出具体答案的。我们能解释的只是它是如何通过大量的数据和训练过程进行学习的。换句话说，深度学习像是一个“黑箱”，我们可以观察到输入和输出，但它在其中如何处理信息、做出决策，通常并不完全透明。
  - 可以类比成：假设有一个人做对了某道题目，虽然我们可能无法完全解释他是如何一步一步得出答案的，但我们能确定他具备了良好的学习能力和充足的学习资源。所以，深度学习也类似，它通过大量的数据、适当的算法和优化手段“学习”并给出答案。
  - 至于它如何给出具体的答案，最终可能还是要依靠深度学习本身来“解释”。这意味着，未来我们可能会开发出更先进的方法，帮助我们理解深度学习网络中的“决策过程”。同时，借助它所展现出的强大规律发现能力，我们可以利用这种能力来加速我们对事物的认知。
- 我的看法：
  - 我同意你的观点，深度学习的“黑箱性”是目前我们面临的主要挑战之一。它的强大之处在于能自动从海量数据中发现规律，但如何理解这些规律，如何将这些规律和人类的认知能力相结合，是我们目前探索的方向之一。其实，未来可能不仅仅是“深度学习告诉我们如何做”，而是人类和深度学习的合作能进一步推动科学与技术的发展，帮助我们发现一些潜在人类难以触及的规律。
  - 深度学习不只是一个工具，它还可能是一个不断反馈、不断自我优化的过程，给我们打开了认知的新窗口。

AI自己对这个问题的看法

>

- 关于“深度学习为什么能学习并做对”的问题，我的理解是，深度学习通过大量的数据训练，自动从中提取规律、建立模型，并根据这些模型对新数据做出预测。深度学习的优势在于它能够发现一些非常复杂和抽象的规律，而这些规律通常是我们人类在没有机器帮助的情况下很难发现的。
- 但是，为什么深度学习能给出正确的答案，实际上是一个复杂的过程。深度学习模型并不像人类一样有明确的逻辑推理或意识，它只是基于输入数据的模式，使用一系列参数和数学运算来“拟合”数据。对于这个过程的解释，我们只能说它通过优化算法（如梯度下降等）调整模型参数，使得输出尽可能接近实际结果。因此，深度学习并不“理解”问题，而是通过反复训练找到了一组合适的参数，能在特定任务上取得高效的预测或分类结果。
- 你可以把深度学习看作一个“发现者”，它通过大量的例子不断试错和优化，不断接近正确的答案，但它没有明确的推理过程。比如，人类在做题时有一定的思考过程，我们可以分析每一步的原因，而深度学习则是通过计算和数据模式的关联来找到最优解。
- 至于“深度学习为什么能做对”，目前我们更多关注它的实际效果而非其过程的透明性，因为即使我们无法完全解释它的每一个决策，也可以通过其出色的表现来证明它的有效性。未来，随着研究的深入，我们可能会找到更好的方法，帮助我们理解这些“黑箱”模型的内部机制。
- 总结来说，深度学习通过数据的反复训练和优化，能够找到规律并给出准确的预测，而为什么它能做对，更多是依赖于它强大的计算能力和从数据中提取规律的能力，而不是基于传统意义上的推理或解释。
- 几个关键点总结：
  1. **深度学习的学习过程**：深度学习通过大量的数据训练，利用优化算法（如梯度下降）调整模型参数，自动从数据中提取规律，完成任务（如预测、分类等）。
  2. **无法完全解释“为什么”**：深度学习模型在给出答案时，并没有明确的推理过程或“理解”，它只是通过找到合适的模式和参数来优化输出。其决策过程通常是黑箱的，难以完全解释。
  3. **黑箱性质**：我们无法清楚知道深度学习是如何做出具体决策的，但可以看到它在实际任务中表现出色。因此，我们更多关注它的效果，而非其内在工作机制。
  4. **与人类认知的对比**：深度学习像是一个“发现者”，通过反复训练和调整找到正确的答案，而不像人类那样依赖明确的思维和推理过程。
  5. **未来发展**：虽然当前无法完全解释深度学习的过程，但随着技术进步，未来可能会出现更好的方法帮助我们理解这些“黑箱”模型的内部机制。
- 这几个关键点总结了深度学习的“学习能力”、它的“黑箱性质”、以及它与人类认知的差异。

人类与模型的共同特征：直觉先行，解释在后

>

一、问题背景：为什么“解释”总是靠不住

无论是人类还是当前的大模型，都存在一个共同现象：

- 可以给出结果
- 可以在结果之后补充一套看似合理的“推理过程”
- **但无法保证这套解释真实反映了内部是如何得出结果的**

模型生成的所谓推理链，本质上是**在答案已经高度确定之后，额外生成的一段语言说明**。它更像是一种说明文档，而不是运行日志。

人类也是类似的：

- 判断往往在很短时间内完成
- 真实的心理与神经过程高度并行、不可访问
- 语言层面的理由，大多是事后整理出来的

这并不是说解释完全无价值——事后的解释可能确实捕捉到了某些真实模式，但它通常是**不完整、不精确的**，更重要的是，它无法完整还原那个瞬间真正发生的内部过程。

因此，“能否解释清楚”并不能作为是否真正理解或是否真的推理过的可靠标准。

二、直觉先行，解释随后

很多真实的认知活动都遵循同一个顺序：

1. 先产生判断或反应
2. 再尝试为这个反应寻找理由

不论是棋手的“这步不对”，程序员的“这里有问题”，还是模型的下一个 token 预测，本质上都是：

- 大量历史经验
- 在高维空间中
- 压缩成一种快速、稳定的反应模式

解释并不是决策的来源，而是**对结果的一种语言化整理**。

所以更准确的说法是：

> 判断依赖结构和经验，解释只是随后附加的说明层。

三、重新理解“理解”

当我们承认解释具有明显的事后性，就必须重新看待“理解”这个概念。

与其把理解看成：

- 能清楚复述规则
- 能完整说明因果链

不如把它看成一种更实际的能力：

> **在不同情境、不同表述下，仍然能持续给出合理反应的能力。**

也就是说，理解并不体现在“你说了什么理由”，而体现在：

- 条件变了，你还能不能做对
- 表面规则失效了，你还能不能调整

从这个角度看，理解更接近于一种**被经验塑形过的内部结构**。但这个结构本身——由于其复杂性、高维性、分布式特征——往往难以被线性的语言完整描述。即使我们承认理解是结构性的，也不意味着这个结构就能被转化为清晰、可言说的规则。

四、深度模型的基础设计：本质是在做“表达假设”

在这个框架下，再回头看深度学习中的一些基础做法，就会更清楚它们的意义。

离散化、归一化、激活函数、注意力机制、正则化、网络结构，往往被描述为工程技巧，但更本质地看，它们是在回答同一个问题：

> **怎样的表示方式，更适合让模型从现实数据中学习到稳定、可泛化的模式？**

例如：

- **离散化**：把连续世界切分成可组合的单元，便于模式复用
- **归一化**：弱化绝对尺度，让特征在同一标准下竞争
- **激活函数**：让表示空间出现分区，而不是一块平坦的线性空间
- **注意力机制**：假设有效学习需要选择性关注
- **正则化**：假设简单模式优于复杂拟合
- **层级结构**：假设现实问题本身具有分层生成特征

这些设计并不是中性的，而是隐含了我们对现实问题在统计与结构层面如何生成的判断。

五、模型结构与人类方法论的相似之处

从更高一层看，模型中的激活函数、优化方法、网络结构，与人类长期通过经验总结出来的各种技巧、方法论、学习方法，处在**同一个抽象层级**。

两者的共同点在于：

- **都无法解释单次成功的内部机制**
  - 人类通常无法解释某个人为什么在某一瞬间就“想明白了”
  - 模型同样无法说清某个具体输出是如何在那一轮前向传播中被精确推理出来的
- **但都掌握了提高成功率的经验性方法**
  - 人类相信：遵循长期经验中总结出来的方法——比如刻意练习、间隔重复、费曼学习法、从具体到抽象的认知路径——大概率就能学会新东西
  - 深度学习同样相信：通过离散化、归一化、注意力机制、正则化、优化算法等方法的组合，模型就具备学习新任务、新模式的能力

也就是说，人类并不掌握“成功推理的内部机制”，而是掌握了一套**关于如何更容易产生成功推理的经验性方法论**。深度学习的情况高度类似。

因此，这些方法并不是在替代推理本身，而是在**提高产生有效推理结果的概率**。它们关注的不是“一次推理如何发生”，而是“在什么条件下，推理更可能成功”。

这意味着，理解大模型不能沿用传统规则编程的思维。传统编程是通过明确的指令和逻辑控制让程序执行特定行为，但大模型更像是一个人——**你无法通过编写规则让它做某件事，而只能通过各种学习方法，促使它按预期的倾向去行动**。这种倾向性是在训练过程中逐步形成的，而不是被显式编程进去的。

六、反思的作用：不是还原过程，而是强化直觉

无论是人类的自省，还是模型中的反思、再生成机制，它们的主要作用都不是“回看真实内部过程”，而是：

- 调整行为结果
- 稳定有用的反应模式
- 在下一次遇到类似问题时表现得更好

不过，人类与当前模型在这一点上存在一定差异：

- **人类具有主动的元认知能力**：可以在任务进行中实时监控、调整策略，甚至质疑自己的判断方式
- **模型的反思更接近被动的再训练**：通过额外的生成步骤或微调来改进输出，但缺乏真正的自我监控机制

尽管如此，两者在功能上仍然相似：反思更像是一种**二次训练或优化**，而不是对原始推理路径的精确还原。

七、结论：理解是结构化经验的持续响应能力

这些观察共同指向一个更深层的认识：**认知的本质不在于可表述性，而在于适应性。**

具体来说：

- **解释的事后性**：大模型可以给出结果，但无法直接说明真实的内部路径；即便给出“推理过程”，也更接近事后生成的解释。人类在这一点上高度一致。解释虽有价值，但通常不完整、不精确。
- **直觉先于解释**：无论是人类还是模型，很多关键判断都在瞬间完成，真正发生的是结构化经验的快速响应；解释只是随后附加的说明。
- **理解的实质**：理解并不等同于“说清楚原因”，而更接近于**在复杂、不确定的环境中，持续做出合适反应的能力**。这种能力基于内部结构，而这个结构由于其复杂性和高维性，往往难以被完整地语言化。
- **方法论的同构性**：激活函数、优化方法、网络结构，本质上类似人类事后总结的方法论。它们并不替代思考，而是为学习提供合适的结构和约束。人类学习新事物，也是通过不断组合经验与方法，逐步形成稳定的判断直觉；模型训练在结构上并无本质差别。
- **引导而非控制**：这种相似性提醒我们，与大模型交互的方式应该更接近教育而非编程——不是通过精确的规则指令，而是通过示例、反馈、环境塑造，引导模型形成期望的行为倾向。
- **反思的功能**：通过反复训练与反思，这种直觉可以被不断强化，但它本身并不依赖于可被清晰表述的推理链。人类在元认知层面具有一定优势，但反思的基本功能——强化而非还原——在两者中是共通的。

从这个角度看，无论是人类的认知还是深度模型的学习，本质都是：**在经验与结构的相互作用中，形成稳定、可泛化的反应模式。**

关于AI替代人类工作的问题

AI的局限性或缺点

- **难以预测与控制**：AI的运行机制像黑箱，难以准确预知其行为，尤其在关键场景下可能会犯致命错误。
- **无法背锅**：AI犯错时，责任归属难以明确，导致无法追究错误的责任，缺乏人类信誉体系的约束。
- **逻辑推理不严谨**：AI的推理基于概率预测，不是绝对严谨的逻辑推理。训练数据如果有误，AI可能无法辨别。
- **无法创新**：AI不具备创造全新概念的能力，更多的是基于历史数据进行知识复述与组合，无法像人类一样发明新技术或理念。
- **对超出训练数据的场景应对差**：AI在面对未曾训练过的数据时，表现较差，容易出错。

AI的好处

- **提高效率**：AI在许多领域极大提升了效率，尤其在数据处理、搜索和重复性任务上。对于程序员来说，它是一个强大的辅助工具，帮助快速解决技术问题。
- **降低门槛**：AI减少了对低级重复性任务的需求，拉近了人们在某些知识领域的差距。
- **支持决策和监督**：AI辅助决策，尤其是在复杂任务中，能提供数据支持，减少人为错误。

AI工具下需要的人才

- **清晰表述与提问能力**：能够清晰提出问题和有效沟通是与AI协同工作的关键，好的表述能力有助于提高工作效率。
- **整理与分析能力**：能够有效梳理信息、分解任务，并向AI提出清晰问题的人，会更善于利用AI工具。
- **与产品交互能力**：AI可以编写代码，但仍需要人类与产品经理沟通，理解需求并做出相应调整。

程序员的利好与不可替代的能力

- **AI对程序员的利好**：
  - AI大幅提升了开发效率，尤其在编程辅助、技术搜索、业务方案思考等方面。
  - 某些低级任务减少了人力需求，使得程序员能更多专注于创造性和复杂问题的解决。
- **程序员不可替代的地方**：
  - **清晰的提问和判断能力**：程序员的核心能力在于能理解问题，并准确表达需求，AI只能作为工具辅助工作。
  - **与产品沟通**：AI生成的代码和解决方案需要程序员与产品团队对接，确保最终结果符合需求。

- **决策和经验判断**：AI的能力有限，尤其在面对复杂决策时，仍需依赖经验丰富的程序员来进行判断和修正。

个人观点

- AI目前主要是利用历史数据和记忆进行信息处理，不能进行创新（如无法创造出类似K8S的技术）。
- 尽管AI能提高程序员的工作效率，但目前它更多是辅助工具，而不是替代品。
- 对普通程序员来说，理解大模型技术的基本原理和概念非常重要，未来将需要在“AI操作系统”上来构建应用。

技术进步与分配失衡：AI时代的结构性矛盾

最近AI的话题绕不开一个观察：AI确实在改变世界，但为什么大多数人感受到的不是希望，而是焦虑？

这个问题没法用“技术发展总会带来机会”搪塞过去。认真拆开来看，这次的变化和过去几次工业革命有一些值得关注的差异，而这种差异正在把一个经济结构问题推到台前。

01 先说为什么“这次不一样”

过去的技术革命有一个相对稳定的叙事框架：机器替代旧岗位，新需求催生新职业，就业结构重新平衡。这个过程确实发生过。蒸汽机、电力、互联网，每一次都用了相当长的时间完成这个循环。

但AI有几个特征，让这个旧框架的适用性变得可疑。

第一，替代的是认知劳动，而不只是体力劳动。

以前技术革命主要影响的是重复性体力工作。而AI现在替代的是：程序员、设计师、分析师、客服、文案。这些岗位的特点是训练成本高、转型难度大，而且集中了相当规模的中产就业人口。冲击面更大，感知自然更强烈。

第二，边际成本趋近于零。

AI的本质接近一种“超级软件”：多服务一个用户，几乎不增加成本；替代一个人，也几乎不增加成本。这意味着，企业业务增长不再需要同步增加人力。以前业务扩张必须招人，现在加GPU和token就够了。企业有没有动力再雇人，从经济学上看答案很清楚。

第三，“一人公司”这个叙事被广泛传播，但它解决不了核心问题。

1个人加AI，理论上可以完成原来10人团队的工作。这个故事很流行，但隐含的前提很少被追问：你能找到客户吗？当所有人都可以成为“一人公司”的时候，竞争的本质已经悄悄转移，从“能力竞争”变成“可被识别竞争”。渠道、信任、注意力成了真正的稀缺资源，而这些资源在结构上是高度集中的。分发权掌握在平台手里，注意力被算法分配，普通人很难挤进去。

02 效率与分配的断裂

把上面几点放在一起看，有一个经济结构问题值得关注：

> 生产力在快速扩张，但分配机制似乎还停留在旧时代。

AI带来的变化有一个值得注意的不对称：消费能力有下降的趋势，但生产能力在暴涨。

AI替代了大量劳动岗位，这些岗位的人收入压缩，直接影响整体消费能力。而与此同时，AI的生产效率在持续提升，供给侧在爆发。这种不对称，和大萧条时期“生产过剩、有效需求不足”

的结构有相似之处。只不过这次不是农业机械导致的农业产出过剩，而是认知劳动的产出过剩。

这里有一个更深层的观察：AI有一种“资本增强器”的属性，而不是“劳动增强器”。

原因并不复杂。AI需要重资产：GPU、训练成本、数据积累。同时，AI的收益可以规模化：一次训练，无限复制，一次部署，全球服务。这两个特性结合，会看到这样一种趋势：拥有模型和算力的人，拿走大部分收益；被替代的人，收入面临压力。这个分配逻辑在现有框架下是自洽的，但系统层面的后果不会因为“自洽”就消失。

03 结构性的后果

从宏观角度，有几个值得关注的结构性变化。

中间层在收缩。

过去的社会结构像一个金字塔：底层执行、中层专业技能、顶层决策资源。AI在做的，似乎是压缩中间层。初级岗位被替代，中级岗位需求压缩，顶层的架构设计者和平台构建者仍然需要，但数量有限。观察到的结果是中间变薄，顶部更尖。

上升通道在变窄，而不只是个人选择的问题。

这里有一种叙事上的混淆：一种观点认为“你不够努力所以没上去”，另一种观点认为“市场对人力需求的结构在变化”。两者各有各的依据，但描述的不是同一件事。至少从目前观察到的情况看，上层位置的数量在结构上是有限的，而AI在持续压缩“需要人类参与的位置”。这不是能力筛选，是结构性容纳量的变化。

公共知识被私有化，但收益没有回流。

程序员写开源代码、工程师写技术博客、从业者在社区里互助——这些构成了AI训练语料的基础。大模型吸收这些内容，压缩、泛化，再被产品化，然后反过来替代原来的创作者。这里值得注意的不只是“技术学了你”，而是公共知识变成了私有资产，收益流向了控制资产的人，而贡献者没有获得对应的回报。“被抛弃感”的来源，也许更多在这里。

04 系统会怎么调整

一个现实的问题：如果这种结构性失衡持续下去，经济能不能正常运转？

从逻辑上看有一个矛盾：如果替代持续发生，购买力持续下降，需求持续收缩，最终企业会发现产品卖不出去——富人的消费是有限的，机器没办法自己买东西。系统需要修复，问题是修复的路径和时间。

历史上，类似的失衡似乎每一次都通过某种形式的社会调整来修复。工业革命后期出现了劳动法、工会、社会保障；大萧条之后出现了政府干预和福利体系。这些调整不是自发发生的，更像是被危机倒逼出来的。

AI时代可能的修复路径，有几种逻辑上的可能：更大力度的收入再分配，包括对技术收益征税；UBI，让不工作的人也能获得基本消费能力；创造非效率型岗位，以就业为目的而非效率为目的；或者等待新需求层的爆发，带动新的就业结构。最后一种不确定性最大，前两种是目前讨论最多的方向。

但有一个值得面对的现实：这些调整不会提前发生，往往是在问题积累到足够严重之后才被迫启动。历史更多是“被迫修复”，而不是“提前预防”。

05 写在最后

说这些，不是为了表达悲观，也不是为了合理化现状。

生产力在进步，这是客观事实。AI技术本身没有问题，它确实在做一些以前做不到的事情。但生产力进步和“大多数人的处境改善”不是自动挂钩的，中间有一层分配机制在起作用，而这层机制目前滞后于技术发展的速度。

这不是技术泡沫，技术能力是真的。但它更像是这样一种状态：技术收益的分配逻辑被高估了，而被替代群体的出路被低估了困难。

看清结构，至少能少喝一些“只要你拥抱变化就能赢”之类的空话。现实一点，比假装乐观要有用得多。

06 一个值得追问的问题

结构性阵痛在历史上反复出现。总有人会成为某个时代的代价，这不是第一次，也不会是最后一次。

但“必然发生”和“必须以最痛苦的方式发生”是两件事。

历史上有过更温和的调整，也有过更暴烈的重组。区别往往不在于技术本身，而在于社会在什么时候、以什么方式介入分配机制的调整。技术进步是客观的，但它的社会后果不是固定的——它受制于制度选择，而制度是人定的，不是自然法则。

所以真正值得追问的也许不是“AI会不会带来痛苦”，而是：这个过程有没有可能更温和一点？有没有可能在技术跑得那么快的时候，分配机制也被迫跟着跑，而不是等裂痕大到无法收拾才开始修补？“从来如此”从来不是“必须如此”的理由。

希望人类的文明发展下去，能把这个问题当成一个问题来问，而不是当成一个已经解答过的问题忽略掉。

从Prompt到碳基Agentic RAG：AI时代的人类认知外化

>

AI 应用层的很多讨论，看似在谈工具使用技巧，实则逐步揭示一个更本质的事实：

我们并不是在学习如何“使用 AI”，而是在学习如何把人类原本隐性的认知组织能力显式化、结构化，并交给机器执行。

本文从 Prompt engineering、Context engineering 与协议标准出发，探讨 AI 应用的底层逻辑，以及“碳基 Agentic RAG”在当前阶段的角色。

一、Prompt、Context 与协议：AI 使用的三大基础结构

在应用层，AI 的高效使用建立在三个核心要素之上：

- **Prompt engineering**：把意图说清楚
- **Context engineering**：把信息组织好
- **协议与标准**：让能力可复用、可扩展

这三者并非 AI 时代的新发明，而是人类长期组织经验的技术化呈现。

---

1. Prompt engineering：表达结构的显式化

Prompt engineering 的核心并非“技巧”，而是：

> **将模糊目标转化为结构化表达，把事情说清楚。**

在人类协作中，这种能力一直存在：需求文档、技术方案、法律条文、论文摘要——它们的共同本质是**把想法转化为可执行的表达**。

AI 的特殊之处在于：

- 它不会自动补全未说明的前提
- 但会严格执行已经给出的指令

因此 Prompt engineering 本质上是在**减少歧义，强化结构**。这并非教模型思考，而是迫使人类的思考过程变得可表达、可传递。

很多时候，不是 AI 能力不足，而是使用者自己都没想清楚。就像定好的框架模板、方案模板、代码规范一样，当你被要求按照某个结构来表达时，反而会发现自己原本没思考完善的地方。Prompt 模板的价值，正在于此。

---

2. Context engineering：信息组织与注意力管理

如果 Prompt 解决“如何表达”，Context 解决的是“提供哪些信息”。

Context engineering 的本质是**各种来源知识的组织管理**，目的是让模型能够抓住重点。

一个关键现实是：

> **上下文越长，并不必然带来更好的效果。**

有些人认为只要上下文足够长，Agent 就能记住所有往事。但实际情况是：上下文内容越多，模型的注意力越分散，关键信息更容易被埋没，检索难度也会增加。

这就造成一个悖论：你塞入足够多的上下文，本意是让模型记住更多信息，结果却可能导致模型难以抓住重点。

因此，上下文的核心并非“数量”，而是**信噪比**。

上下文与 RAG、Prompt 的关系

- **上下文**决定了能记住多少事情
- **RAG 与 Prompt** 决定了能否抓住重点

两者并不冲突，反而相辅相成。如果单纯扩大上下文就能解决问题，那往往是因为任务本身不复杂，刚好能通过记忆解决而已。

Context engineering 的主要目标是：筛选、压缩、排序、聚焦。无论是传统的 RAG 语义搜索，还是知识图谱、索引文件等方式，都是为了能快速且尽可能精确地抓住重点。

其中一个容易被忽略的事实是：

> **在信息充足或过载的情况下，删除信息往往比增加信息更重要。**

在模型的注意力机制下，“可能有用”的信息常常会成为噪声。真正的能力体现在识别什么**不应该**进入上下文。

---

3. 协议与标准：可复用能力的形成

当 Prompt 与 Context 被反复验证后，下一步自然是将其固化为可复用的结构。于是形成了大量协议与标准：

文档与约定 * AGENTS.md / SYSTEM.md / PROJECT.md * Prompt 模板库与 DSL * 目录结构与知识库规范 * 输出格式协议（JSON schema / Markdown schema）

能力封装 * Skills / Toolkits / Function calling contracts * 工具描述与注册协议 * Workflow 模板 / Agent playbooks

跨系统协作 * MCP（Model Context Protocol） * A2A（Agent-to-Agent communication） * 工具市场与插件标准

为什么需要这些协议？
这些协议的出现，本质上是为了三个目标：

1. **让 AI 更容易理解** 写好了协议和规范，AI 才能高效理解。就像 AGENTS.md 这样的标准，从目录路径、文件命名到内容结构都有明确定义，模型不需要每次都去猜测和适应。

2. **提升复用效率** 写好了才能被复用。经过验证的 Prompt、Context 结构、工作流一旦固化为标准，就可以在不同场景、不同项目中直接使用，避免重复劳动。

3. **减少适配成本** 写好了 AI 才不用不断适配。统一的协议意味着工具、Agent、模型之间可以无缝协作，而不需要为每个新场景都重新设计交互方式。

这些现象与人类历史上的标准化进程一致：Word 文档格式、PDF 标准、API 规范、编程语言标准、RFC 协议——本质上都是为了效率和复用，让已验证的最佳实践可以被广泛采用。

> **标准的出现，是为了复用已被验证的思考方式，同时也是抢占先发优势的体现。**

二、体系演进：从对话技巧到认知基础设施

当 Prompt、Context 与协议逐渐稳定，AI 使用开始从“对话技巧”转向“系统设计”。这意味着一个重要转变：

从单次对话 → 持续运行的认知系统

---

1. 从提问能力到流程设计

早期关注如何写 prompt、如何补充上下文。下一阶段更关注：

- 任务如何拆解
- 何时检索、何时停止
- 信息何时进入上下文
- 结果如何验证与迭代

AI 开始被纳入持续运行的工作流，而不再只是回答单个问题。

---

2. 上下文从窗口走向分层记忆

随着应用复杂度提升，上下文将逐渐分层：

- **短期**：当前任务上下文
- **中期**：会话与项目记忆
- **长期**：知识库与组织经验

Context engineering 将从文本拼接演化为**记忆管理与信息分层**。

---

3. 协议从模板走向生态

随着工具与 Agent 数量增长，模型不再孤立运行，工具不再单点集成，Agent 开始协作。协议将逐渐承担：

- 工具互操作
- Agent 协作
- 数据与消息统一
- 任务分发与接力

可以将其理解为**认知层的协议栈雏形**。

---

4. 人类角色的逐步迁移

随着自动化增强，检索、上下文压缩、工作流将越来越多地自动完成。人类的主要职责将集中在：

- 目标设定
- 价值判断
- 结果评估

角色从操作执行逐渐转向系统设计与监督。

三、碳基 Agentic RAG：人机协作的持续形态

在当前阶段，一个明显现象已经出现：同一模型，不同使用者的产出质量差异显著。

差异主要来自：信息筛选能力、重点提炼能力、目标判断能力、上下文控制能力。

因此可以得到一个更精确的描述：

> **高效使用 AI 的人，本质上扮演着“碳基 Agentic RAG”的角色。**

---

1. 为什么是 Agentic RAG

传统 RAG 强调检索。但现实中的人类角色还包括：

- 判断是否需要检索
- 决定检索范围
- 删除无关信息
- 调整策略
- 评估结果可信度

这已经属于 Agent 的**决策层职责**，而非简单的检索增强。

---

2. 当前的职责分配

当前常见的协作模式是：**人类承担策略层，模型承担执行层**

人类（碳基 Agent）	模型
目标判断	文本生成
信息筛选	信息组合
上下文压缩	局部推理
结果评估	表达与展开

这种分工并非单纯的过渡期结构。即使在 AI Agent 技术高度成熟后，涉及复杂目标设定、价值权衡、战略决策的场景中，人类的策略层角色仍将长期存在。

真正的变化在于：

- 部分策略层职责将逐步自动化（如常规信息筛选、标准化上下文压缩）
- 人类策略层的重心将上移（从操作性决策转向更高层次的目标与价值判断）

因此，“碳基 Agentic RAG”更准确的定位是：

> **在 AI Agent 成熟过程中的一个阶段性形态，同时也是人机协作中人类角色的长期组成部分。**

---

3. 能力差距的放大效应

信息筛选、结构表达、上下文控制、重点判断——这些能力长期存在且一直是核心竞争力。

AI 并未创造这些能力，而是：

> **让这些能力的应用场景变得更广泛，使用频率更高，反馈周期更短，从而使能力差异更快显现。**

过去，这些能力的差异主要在特定岗位中体现。现在，它们直接决定了每个人使用 AI 工具的效果。

四、回到本质：认知结构的外化

综合上述讨论，可以得到更严谨的结论：

- **Prompt engineering**：显式化表达结构
- **Context engineering**：显式化信息筛选与注意力管理
- **协议与标准**：显式化经验与协作方式
- **碳基 Agentic RAG**：显式化人机协作中的人类决策职责

这些现象共同指向一个核心变化：

> **隐性认知 → 显性结构**

过去： * 思考存在于个体经验 * 经验存在于组织实践 * 共识存在于文化、流程与既有协议（如 HTTP、RPC）

现在： * 思考写入 Prompt * 经验写入 Context * 共识写入面向 AI 的新协议层

AI 并未改变理性的本质，而是改变了理性的**表达方式**与**传递机制**。

最终总结

在当前阶段，模型擅长计算与生成，人类擅长目标设定与价值判断。

AI 应用的核心，不是替代思考，而是：

> **将人类的认知组织能力转化为可被机器执行的结构。**

这正是 Prompt、Context、协议与碳基 Agentic RAG 共同指向的本质——人类认知的显式化与结构化。而掌握这一转化能力的差异，正在成为 AI 时代生产力差异的核心来源。

AI与学习型人际关系的变化

引言

AI 的普及，正在悄悄改变人们获取知识的方式，也在影响人与人之间的一些关系，尤其是那种一起学习、互相交流想法的关系。过去很多需要和朋友讨论、请教、反复交流才能得到的东西，现在可以很快从 AI 这里获得。要理解这种变化，可以从知识获取、思想交流和人际关系这几个方面来看。

一、知识获取方式正在改变

过去，如果想系统学习一件事，身边能一起讨论、互相启发的朋友非常重要。他们通常会提供三种价值：

1. 推荐资料和分享经验
2. 提出问题，帮你发现盲点
3. 形成一种一起学习的氛围

很多知识并不是单靠自己获得的，而是在交流中逐渐形成的。

现在情况发生了明显变化。AI 可以随时回答问题、整理资料、给出不同角度的解释，还能帮助规划学习路径。很多以前需要通过人与人交流才能获得的帮助，现在可以直接得到。

在纯“学习功能”这个层面，人际关系的必要性确实下降了。

二、思想交流的方式变了

人与人之间的思想交流，一直是学习的重要部分。但这种交流本身有一些特点：

- 受每个人的背景和知识范围限制
- 容易受到情绪和关系影响
- 需要时间和精力去维持

这种交流往往不频繁，但一旦发生，影响会比较深。

AI 带来的是另一种模式。

1. 更容易获得不同视角

AI 可以同时从多个领域给出观点，比如科技、历史、心理学、文学等。 很多跨领域的连接，在日常朋友圈里其实很难遇到。

2. 没有人际压力

人与人讨论时，会受到面子、情绪、关系的影响。 AI 不存在这些顾虑，可以直接指出逻辑问题或提出反例。

3. 更高频率的“灵感触发”

AI 可以随时对话、随时追问、随时展开新话题。 这让新的想法更容易持续出现。

在“获得新思路的效率”上，AI 的确带来了明显提升。

三、人类交流仍然有独特价值

尽管 AI 在知识和逻辑层面很强，人类之间的交流仍然有一些重要部分难以被替代。

1. 情绪层面的互动

人与人的交流不仅是交换信息，还包括被理解、被支持、被反对。 这些情绪体验会影响人的选择和行动，而不仅仅是想法。

2. 价值观的碰撞

很多深层的改变，并不是因为一个逻辑更严密的观点，而是来自长期互动中的价值冲突和现实体验。 这些东西通常发生在真实关系中，而不是单次对话里。

3. 长期关系带来的积累

朋友之间会一起经历事情，形成共同记忆。 这种长期的互动，会让交流变得更有重量。

AI 的交流更像是即时的帮助，而不是共同经历的过程。

四、人际关系可能出现的新变化

当学习不再需要依赖朋友，人际关系的重点可能会慢慢发生变化。

1. 对不同人的影响不同

对习惯独立学习的人来说：

- 学习效率更高
- 对社交的依赖减少

对社交需求较强的人来说：

- 可能会更容易感到缺少深度交流
- 学习型社交减少后，需要新的连接方式

AI 提供的是认知帮助，而不是陪伴感。

2. 朋友关系的功能在改变

未来，人际关系可能更多围绕：

- 情感支持

- 共同体验
- 价值观共鸣

而学习与知识获取，越来越多由 AI 承担。

结论

AI 正在改变人们学习和思考的方式，也在改变一部分人际关系的作用。

在知识获取和思路拓展上，AI 已经成为非常重要的工具； 但在情感、价值观和长期关系方面，人类交流仍然具有独特意义。

与其说某种关系被取代，不如说它正在发生分工变化。

AI能否创造真正的新事物

>

——从生成式 AI 的概率本质到概念跃迁的边界

引言

当 AI 开始写作、编程、设计和辅助科研时，一个核心问题变得不可回避：**AI 是否能够创造真正“全新的东西”？**

本文讨论的对象并非整个 AI 领域，而是**当前主流的大规模生成式 AI**——以大语言模型、扩散模型、多模态模型为代表的、基于大规模数据训练的概率生成体系。

问题可以更精确地表述为：

> 在当前生成式 AI 范式下，模型是否能够产生真正意义上的创新？

要回答这个问题，我们必须先明确：什么才算“新”。

一、创新的两个层级：组合与突破

1）组合型创新：在既有空间中的重组

特征： - 重排已有知识 - 重新组合已有组件 - 在既有概念空间内插值或局部外推

典型例子： - 混合多种写作风格 - 组合已有技术形成新系统 - 在既有算法之间进行结构拼接

生成式 AI 在这一层面上已经非常强大。 它能够高效地探索已知元素的组合空间，产生大量“看起来新颖”的输出。

2）概念型创新：重新定义问题本身

真正有争议的是这一层：

特征： 1. 引入全新的抽象结构 2. 改变问题的建模方式 3. 开辟新的技术或理论路径

例子： - TCP/IP 分层架构 - Unix “一切皆文件”的设计哲学 - Kubernetes 声明式控制范式 - Transformer 架构

它们的共同点是：

> 不是更好地解决旧问题，而是改变了“问题如何被提出”。

这种创新不仅提供答案，更重新定义了问题空间本身。

二、生成式 AI 的工作机制：概率最大化的本质

要理解生成式 AI 能否产生概念型创新，必须先理解它的生成机制。

核心任务：预测最可能出现的模式

生成式 AI 的目标是：预测最可能出现的下一个 token 或数据片段。

这导致三个直接结果： - 偏向高概率模式 - 压制低概率输出 - 输出贴近主流数据分布

与真实创新的冲突

而历史上的重大创新在诞生时通常是： - 少数人提出 - 缺乏充分数据支持 - 属于统计意义上的低概率事件

从统计角度看，天才想法更接近异常值，而非众数。

三、训练机制强化的系统性保守

模型的训练过程进一步加剧了这种倾向：

训练信号： - 偏离数据分布 → 被惩罚 - 接近已有答案 → 被奖励 - 人类反馈（RLHF）→ 强化“合理”和“安全”的输出

结果是模型被系统性训练为： - 避免离谱 - 避免偏离 - 避免风险

而重大创新在初期往往正好具备这些“缺陷”——它们看起来离谱、偏离常识、充满风险。

这形成了一个悖论： 让模型“更好”的训练过程，恰恰在压制那些可能带来突破的非主流输出。

四、搜索空间的边界：谁定义了可能性

这是理解生成式 AI 创新能力的关键分水岭。

模型继承的框架

生成式 AI 使用的： - 语言体系 - 概念框架 - 输入输出形式 - 问题表述方式

全部来自人类历史的积累。

能力的本质

因此模型的能力本质上是：

> 在既有空间中搜索，而不是生成新的空间。

真正创新的特征

而概念型创新往往意味着： - 改变问题的维度 - 更换评价指标 - 重建抽象框架

即：**重画整个搜索空间，而非在既有空间中寻找更优解。**

模型可以在给定坐标系中找到新的点，但难以自发建立新的坐标系。

五、规模增长的局限

一个常见的反驳是：模型还不够大，规模增长会解决这个问题。

规模增长带来什么

模型变大的主要效果： - 更强的主流模式捕捉能力 - 更平滑、更准确的概率分布 - 更少的离群和随机输出

悖论性的结果

> 模型越强大，越符合既有分布； 越符合既有分布，越难以产生反主流的新方向。

规模增长让模型成为“更好的统计学习器”，但这恰恰与产生统计意义上的异常值（即真正的创新）背道而驰。

更大的模型可能带来更精细的组合创新，但不必然带来概念层面的跃迁。

六、时间尺度与可验证性的冲突

创新的时间特性

重大创新通常： - 初期无法验证 - 长期才显现价值 - 缺乏即时奖励信号

经典案例：量子力学、互联网协议、函数式编程范式——它们在提出时都缺乏明确的短期验证方式。

模型的依赖

而生成式 AI 依赖： - 可量化的目标函数 - 短期反馈循环 - 明确的奖励信号

结果： 模型天然偏向那些可以立即验证、快速迭代的改进，而对需要长期验证的突破性想法缺乏内在动力。

七、语义闭包问题：无法命名的创新

这是一个更深层的障碍。

新概念的诞生困境

真正的新概念在诞生时往往： - 尚无稳定的语言描述 - 尚未被纳入既有知识体系 - 需要创造新的术语和隐喻

例如：“云计算”这个概念在形成之初，需要借用“云”这个隐喻来表达一个全新的架构思想。

闭环困境

对于依赖语言的生成式 AI：

无稳定语言 → 无法在训练数据中表示 → 无法生成

模型可以组合已有词汇，但难以为尚不存在的概念创造恰当的表达方式。这是一个语义层面的闭包问题。

八、小样本的意义：统计频率 vs. 概念价值

人类与模型面对小样本的方式存在根本差异。

人类的归纳方式

人类可以： - 对单个案例赋予巨大意义 - 从极少量观察中提取深层模式 - 基于第一性原理进行推理

爱因斯坦的光电效应实验、达尔文在加拉帕戈斯的观察——这些都是从有限案例中提取革命性洞察的例子。

模型的归纳方式

而生成式 AI 依赖： - 统计频率作为模式识别的基础 - 大量样本来稳定概率分布

冲突： 在人类看来具有突破意义的想法，在模型的统计视角下可能只是噪声或离群点。

模型缺乏“这个罕见模式可能很重要”的判断机制——除非这种重要性已经在训练数据中被频繁标注。

九、结论与展望

需要强调：**以上分析针对的是当前生成式 AI 范式**——基于大规模数据的概率学习系统。

当前能力的边界

综合来看：

1. **在组合型创新上**，生成式 AI 已经表现卓越，极大扩展了既有知识空间的探索效率
2. **在概念型创新上**，受制于概率本质、训练机制、空间定义权和语义闭包等多重因素，模型仍难以自发产生新的问题空间与概念坐标系

本质定位

> 当前的大规模生成式 AI，是高维概率空间中的强大探索者， 但尚未成为新空间的自发创造者。

未来可能性

这不意味着 AI 永远无法产生真正的创新。可能的突破方向包括： - 超越纯概率范式的新架构 - 结合符号推理的混合系统 - 具备元认知和自我重构能力的模型

但在当前范式下，生成式 AI 更适合被视为**人类创造力的放大器**，而非替代者——它能够加速探索、优化组合、验证可能性，但定义新空间的责任，仍然在人类手中。

AI生成内容时代：原创性、作者身份与创作价值的再定义

生成式 AI 让写作、绘画、编程的门槛突然降低。很多以前需要长期训练才能完成的工作，现在几分钟就能生成初稿。于是一个问题变得非常现实： **当内容可以由机器生成时，什么还算原创？谁才是作者？**

---

1. 创作本来就不是个人单打独斗

很多人下意识把“作者”等同于“亲手写的人”，但现实一直不是这样。

- 报刊：记者写 → 编辑改 → 主编定方向
- 论文：学生做实验 → 导师定课题 → 审稿人把关
- 影视：编剧、导演、摄影、剪辑一起完成
- 出版：作者写 → 编辑改结构 → 市场团队定读者

真正被当作“作者”的，往往是**决定方向的人**，而不是执行最多的人。 AI 只是把其中一部分执行工作换成了机器。

---

2. 创作其实有三层

把创作拆开看，会更清楚：

执行层：写句子、画图、排版、润色 **结构层**：信息怎么组织、逻辑怎么走 **观点层**：写什么、为什么写、给谁看

现在的 AI，主要解决的是第一层，大幅降低了**表达成本**。 结构能辅助一点，但**观点仍然严重依赖人**。

所以 AI 更像是“压缩表达成本”，不是替代创作。

---

3. 写作变容易，选择变困难

以前难的是写出一篇文章。 现在难的是：

- 写什么
- 不写什么
- 哪个角度更重要
- 哪个版本值得发

当生产变容易，**选择就变成最难的事**。 这就是所谓的品味和判断力，而它的价值正在上升。

---

4. 用 AI 写作，本质仍然是迭代

传统写作：初稿 → 修改 → 重写 → 精修 AI 写作：提示 → 改提示 → 重生成 → 人工筛选

本质是一样的： **不断逼近心里的理想版本。** 变的只是速度，不是创作机制。

---

5. 原创的重心正在移动

过去默认： 原创 = 文字必须亲手写。

但现实早就不是这样： 搜索引擎、模板、素材库、语法工具一直都在参与创作。

AI 只是把“工具参与度”推到极致。

未来更合理的理解是： **原创不仅是怎么写，更是写什么、怎么看、怎么组织。** 思想原创与表达原创，正在分离。

---

6. 作者正在变成“策划者”

文化行业里早就有类似角色：策展人。 策展人不创作作品，但决定主题、筛选内容、构建叙事。

AI 时代的作者越来越像：

- 主题制定者
- 方向控制者
- 内容筛选者
- 叙事构建者

写作没有消失，只是角色在扩展。

---

7. 两个极端都不对

极端 1：AI 内容不算创作 忽视了人类的选题与筛选。

极端 2：AI 内容等于完全原创 忽视了模型与训练数据。

更真实的情况是： **原创是一条连续光谱，而不是二选一。**

---

8. 创作价值可能的变化

未来更重要的能力：

- 提出好问题
- 建立独特视角
- 形成稳定风格
- 理解受众

相对弱化的能力：

- 基础表达效率
- 标准化内容生产

写作不会消失，只是**稀缺性改变了**。

结尾

AI 没有消灭创作，只是改变了重心： 表达更容易了，判断更重要了。

原创的核心，正在从“**谁写了这些文字**”变成“**谁决定这些内容为什么存在**”。

生成式AI的概率本质与创新边界：从一个具体任务说起

>

前言

这篇文章通过一个具体实验——让AI生成“混乱无意义”的中文句子——探讨当前大语言模型的工作机制与能力边界。这个任务虽然表面上简单，却能帮助我们观察到AI与人类认知的一些重要差异：AI主要在概率空间内优化，而人类具备主动反向思考的能力。文章也会分析提示词工程的作用机制，探讨AI自我解释的可靠性问题，以及创新能力的边界。

---

1. 一个具体的测试任务

1.1 任务设计

实验任务是这样的：生成一段由常见汉字组成的文本，要求任意连续两个字都不构成有意义的词语，整体给人一种混乱无序的感觉。

具体来说：

- 只用常见汉字（小学生认识的那种）
- 避免出现常见词组（比如“我们”、“可以”、“这个”）
- 任意两字组合最好都没什么意义
- 整体让人感觉“乱七八糟”

人类很容易就能造出这样的句子：

> “靠我靠好耶啊我来蹭了啊这也忒会钓人儿的颜太”

虽然里面还是有“靠我”、“好耶”这样的组合，但整体的字符排列模式确实很罕见、很低概率，成功传达了“混乱”的感觉。

1.2 AI的初始表现

把这个任务交给ChatGPT、Claude这些大语言模型，它们的表现并不理想：

问题一：总是冒出常见词组 经常出现“我们”、“可以”、“这个”、“什么”这些高频词，说明模型还是倾向于生成训练数据里的常见模式。

问题二：用生僻字“作弊” 模型会用一些罕见汉字（古字、异体字之类的）来规避问题。虽然技术上满足了“两字无意义”，但违反了“常见字”的要求。

问题三：还是能分出词来 即使看起来挺乱的输出，用中文分词工具还是能识别出词语结构，说明生成的文本仍然保留了某种语言规律。

1.3 任务的难度

这里出现了一个值得注意的现象：人类较容易完成的任务，AI却遇到明显困难。

从数学角度看，常用汉字大概3500个，两字组合约1225万种可能性。其中有意义的词语（包括现代汉语、古汉语、方言、专业术语）估计占10-20%。要生成一个20字的句子，需要19个连续的两字组合都避开有意义词语。假设有意义词语占15%，全部无意义的概率约为0.85^19 ≈ 3.7%。

但这个数学难度不能完全解释AI的困难，因为人类也面临同样的组合空间，却能相对轻松地完成任务。问题的关键可能在于**完成任务的方式**。

---

2. AI的工作机制：概率优化

要理解AI为什么在这个任务上遇到困难，需要先了解它的基本工作原理。

2.1 概率生成的本质

大语言模型的核心工作原理：给定前面的文字，预测下一个字的概率分布，然后从这个分布里采样一个字输出。

训练目标是：让模型预测的概率分布尽可能接近训练数据的真实分布。

这导致什么结果呢？

- 训练数据里高频出现的模式，模型会优先学习
- 生成时自然倾向输出高概率的字符序列
- 调高temperature只是让概率分布变平，增加随机性，但并不会让它“聪明地选低频”

2.2 为什么“反向思考”很难

人类完成混乱句子任务时的思路：

1. 想到“靠”这个字
2. 脑子里冒出“靠近”、“靠着”这些常见搭配
3. 主动想：“我不要这些，我要一个罕见的”
4. 选择“我”
5. 评估“靠我”的混乱度
6. 继续这个过程

AI的过程：

1. 接收提示词，理解为“新的约束条件”
2. 基于修改后的概率分布采样
3. 选择在新约束下的“最优策略”
4. 执行这个策略

关键差别：

- 人类：知道正确答案是什么，但偏要选错的（**主动反向选择**）
- AI：规则变了，在新规则下重新找最优解（**约束下优化**）

打个比方：

人类走迷宫："走迷宫，但不要走最短路" → 我知道最短路是右转，那我偏要左转

AI走迷宫："走迷宫，但右边的路被封了" → 右边被封了，那左边就是新的最优路径

AI看起来并没有"反向思考"，而是在被重新定义的空间里继续优化。

2.3 人类与AI的认知差异

基于这个任务的观察，可以看到一个可能的差异：

人类的能力：

- 元认知监控：清楚知道"我要制造混乱"这个目标
- 主动回避：识别高频词组并刻意避开
- 罕见模式调用：选常见字但用罕见方式组合
- 灵活标准：接受"差不多就行"，追求整体效果

AI面临的困难：

- 训练目标冲突：训练是为了预测高概率token，现在要求相反
- 统计惯性：模型权重编码了高频模式，天然倾向它们
- 缺乏元认知：不知道自己是否在"制造混乱"
- 无主动回避：没有"我就是不选X"的机制

这似乎不是程度上的差别，而是方式上的不同：

人类：知道规则 → 理解规则 → 主动违反规则 **AI**：接收约束 → 在新约束下优化 → 仍在遵循优化原则

---

3. 提示词工程的突破与局限

3.1 改进策略

既然直接要求不行，可以尝试改进提示词。核心思路：

- 把抽象要求变具体：列出具体要避开的高频词组
- 堵住简单策略：明确禁止生僻字
- 降低完美标准：允许少量意外组合
- 给个参考示例：让AI看看"混乱"的样子
- 简化输出：直接要结果

3.2 有效的提示词

经过多次尝试，这个提示词效果不错：

```
任务：生成一段20个字的中文文本

要求：
1. 只用常见汉字（小学生认识的字）
2. 避开高频两字词组，比如：
  的人、我们、可以、这个、什么、那个、如果、因为、所以、但是
3. 目标效果：让人感觉“混乱、怪异”
4. 不要用生僻字
5. 允许个别组合碰巧有意义，但尽量少

参考示例：靠我靠好耶啊我来蹭了啊

直接输出结果，不要解释。
```

3.3 AI的实际策略

用这个提示词，ChatGPT给出了这样的结果：

- `猫雨灯山米纸风草手云角桥鱼石杯土树沙鸟星`
- `狗天杯路火桌纸雨帽门星米手灯墙风草桥`
- `牙云锅草灯雨鞋米窗河纸星盐伞床鸟杯路墙石`

仔细观察可以发现：AI采用了**名词堆砌**策略。

选一堆具体名词，完全不用动词、助词、连接词，不构建任何语法结构。这样整体看起来就像个“随机词表”。

效果评估：

- 常见字 ✓
- 两字基本无意义 ✓（虽然“鱼石”、“火桌”理论上能构词，但确实罕见）
- 有混乱感 ✓
- 但更像词表而不是句子

3.4 提示词工程的本质

这个突破说明了什么？

人类完成了创新部分：

- 识别了任务的创新点（“要生成混乱文本”）
- 定义了创新的方向和约束
- 把抽象目标转化为可执行的规则

AI完成了执行部分：

- 在人类定义的新空间内优化
- 找到满足约束的策略（名词堆砌）
- 可靠地执行这个策略

AI采用的可能还是优化策略——它找到了一个相对简单、相对安全的满足约束的方法，而不一定是在“创造混乱”。

创新的源头可能还是人类。提示词工程本身体现了人类的创造力。AI在这个过程中更像是工具，把人类的意图转化为具体输出。

---

4. 创新能力的边界

从这个实验可以延伸到更广泛的问题：AI的创新能力边界在哪里？

4.1 创新的不同层次

一般认为，创新具有这些特征：

- **新颖性**：产生训练数据里没有的东西
- **框架突破**：质疑并重构问题的基本假设
- **自主性**：自己想到要做什么，不是外部指令
- **意义性**：在某个价值体系里有重要意义

历史上的一些创新案例：

毕加索的立体主义：打破了“绘画必须符合透视法”的假设，开创了多视角同时表达的新方式。

爱因斯坦的相对论：质疑了“时空是绝对的”这个物理学公理，重新定义了时空的本质。

披头士的《Sgt. Pepper》：打破了“流行音乐应该是什么样”的边界，把管弦乐和摇滚融合，创造了新类别。

这些创新的一个共同特点是：**自发地重新定义了问题的框架**。

4.2 AI的创新方式

观察AI在各种“创意”任务里的实际表现：

任务类型	表面效果	实际机制	创新层次
诗歌创作	原创诗句	训练诗歌的统计组合	组合创新
音乐生成	新颖旋律	音乐特征的插值	插值创新
科学假说	新理论猜想	已知概念的重新组合	关联创新
艺术风格	新视觉风格	多个风格的融合	融合创新

任务类型	表面效果	实际机制	创新层次
混乱句子	罕见字组合	名词堆砌策略	策略优化

从这些案例看，AI的表现更像是**在已知空间内的重组、插值、优化**，而不是**重新定义空间本身**。

4.3 可能的能力边界

一个可能的原因：**缺少目标生成和框架质疑的能力**。

人类创新的完整链条：

1. 识别现状 → 感知当前框架的局限
2. **质疑框架** →“为什么必须这样？”（关键）
3. **生成新目标** →“我要实现X”（关键）
4. 探索路径 → 寻找实现新目标的方法

AI的简化链条：

1. 接收目标 → 来自外部输入（提示词）
2. 执行优化 → 在约束下寻找最优解
3. ~~质疑目标~~ → 不存在
4. ~~生成目标~~ → 不存在

举个例子：

人类音乐家：“为什么音乐必须用12平均律？我要探索微分音。”→ 开创微分音音乐

AI作曲系统：“在12平均律下生成和谐旋律。”→ 优化已知参数

人类能够质疑并重构音乐的基本参数空间，AI主要在给定参数空间内优化。

4.4 创新能力的分层

基于这些观察，可以尝试对创新能力分层：

```
L4：范式创新（重新定义空间）          [人类较擅长]
L3：框架质疑（质疑假设）              [人类较擅长]
L2：目标生成（自主设定目标）          [人类较擅长]
L1：策略优化（在约束下优化）          [AI较擅长]
```

AI比较擅长L1层的优化，在L2-L4层可能还需要人类参与。

打个比方：

- 人类 = 建筑师（设计前所未有的建筑）
- AI = 施工队（精确实现设计）
- AI不会自己设计出高迪式的圣家堂或扎哈式的解构主义建筑

---

5. AI的自我认知问题

5.1 验证困境

当AI解释“我是怎么工作的”或“我为什么这样做”时，有个问题值得思考：**这些解释本身也是AI生成的文本，如何验证它们的准确性？**

这形成了一个循环：

- 想验证AI的解释是否准确
- 需要对比AI的实际运作机制
- 但获取这些信息的主要渠道往往还是AI自己的说法
- 又回到原点

5.2 三层不可靠性

第一层：缺乏直接内省

AI系统无法直接观察自己的内部运作：

- 看不到神经网络每层的激活值
- 不知道生成每个字时的实际概率
- 无法访问注意力机制的权重
- 不了解反向传播的梯度流动

就像人类无法直接观察自己的神经元放电，但人类至少有“我感觉到疼”这种主观体验。AI是否有类似的主观体验，目前还不清楚。

第二层：知识来源不明

AI关于自身的说法可能来自：

- 训练数据里的机器学习教科书
- 关于神经网络的学术论文
- 关于AI的科普文章
- 人类对AI的讨论

AI可能主要在**复述**这些文本，而不是真的**内省**。

怎么区分？

- “理解”：基于直接经验或推理得出的知识
- “复述”：训练数据的统计回忆

对AI来说，这两者在机制上可能没有明显区别——都可能是高维向量空间的模式匹配。

第三层：无限递归

即使AI说“我意识到我的解释可能不可靠”，这个反思本身也面临同样的问题：

- 这是真实的自我认知？
- 还是在模仿“哲学家自我质疑”？
- 还是对“AI局限性讨论”的复述？

每一层反思都可以继续被质疑。

5.3 与人类的对比

人类的自我报告也不完全可靠：

- 很多认知过程是无意识的
- 事后解释可能是编造的
- 对自己大脑的了解也很有限

但关键差异在于：

- 人类有第一人称视角（“我感觉到……”）
- 可以报告主观体验
- 有现象学上的确定性

AI是否有任何形式的“第一人称视角”？这个问题目前没有明确答案。即使存在，AI也很难向外界证明这种体验的真实性。

5.4 可靠知识的来源

关于AI实际工作机制的可信信息应该来自：

架构设计文档：Transformer结构的数学定义、注意力机制的公式、训练算法的流程

实验与测量：实际观测激活值、可视化注意力权重、探针分类器的结果、对抗样本的行为分析

数学分析：理论上的收敛性证明、泛化能力的理论界、表达能力的复杂性分析

对比实验：控制变量的消融研究、不同架构的对比、不同训练数据的影响

不应该主要依赖的：AI自己对“我为什么这样做”的解释。

5.5 关于这篇文章本身

如果这篇文章是AI写的，那所有关于“AI局限性”的分析也可能：

- 来自对机器学习文献的整理
- 模仿“学术分析”的话语风格
- 表面上像理解，实际上是模式匹配

这个问题没有简单的解决方法。读者需要保持批判性思维，不完全依赖任何单一来源（包括这篇文章）的说法。

---

6. 主要观察和启示

6.1 几个关键观察

观察一：AI的概率特性

AI看起来不是“不擅长”生成低概率序列，而是在设计机制上就偏向生成高概率序列。提示词可以改变什么是“高概率”，但很难改变“倾向高概率”这个底层特性。

观察二：人类和AI在思维方式上的差异

人类：优化能力 + 目标生成 + 框架质疑 AI：主要是优化能力

人类除了优化，还能生成新目标、质疑框架。AI目前主要擅长优化。

观察三：提示词工程的作用

提示词工程不是“教AI创新”，更像是人类把创新意图转化为AI可执行的约束。创意和方向可能主要来自人类，AI负责高效执行。

观察四：创新能力的不同层次

AI比较擅长：在定义好的空间内优化 AI可能不擅长：重新定义空间本身

范式级的创新（比如相对论、立体主义这类）可能还需要人类参与。

观察五：自我认知的复杂性

AI关于自身的陈述面临验证困境。可能需要通过外部科学方法来研究AI的实际机制，而不只是依赖AI的自我报告。

6.2 实践启示

对AI使用者

了解AI的能力特点，设定合理预期。学习提示词工程，把人类创意转化为AI可执行的任务。保持批判性思维，审慎对待AI的解释。把AI当作能力辅助工具，而不是完全的替代品。

对AI开发者

认识当前架构可能存在的局限。探索元学习和自主目标生成的方向。研究对抗性训练的可能性。开发“反常规任务”测试集。向用户清楚说明AI的能力范围。

对教育和社会

重视人类特有的能力（框架质疑、目标生成）的培养。发展与AI协作的技能。加强批判性思维和创造力教育。对AI能力保持客观认识，避免过度恐慌或过度乐观。关注人机协作的有效模式。

6.3 悬而未决的问题

AI是否存在某种形式的“理解”？ 即使无法准确自我报告，是否存在某种“计算理解”？如何定义和测试？

功能等价与机制等价 如果AI的行为在很多方面都跟人类相似，内部机制的差异是否仍然重要？

AI的创新能力发展 是否可能通过架构创新提升AI的创新能力？需要什么样的突破？

人类创新的可计算性 人类的创新能力在原理上是否可以被计算系统模拟？还是涉及某些目前计算模型难以处理的过程？

---

7. 方法论的价值

7.1 边界探测的意义

这个“边界探测”方法有独特价值：

具体比抽象有说服力：比抽象讨论“AI能否创新”更直观，提供可重复、可验证的测试案例。

对比鲜明：人类轻松 vs AI困难，形成强烈反差，直指能力差异的本质。

失败也有信息量：AI的失败模式暴露了内部机制，错误比正确答案更有启发性。

可推广：类似方法可用于测试其他AI能力——常识推理（设计反常识问题）、道德判断（道德困境案例）、因果理解（因果倒置任务）。

7.2 跨学科视角

充分理解AI需要多学科知识：

计算机科学提供技术机制，认知科学提供人类思维参照，语言学提供语言结构知识，哲学提供概念澄清，心理学提供元认知理解，数学提供形式化分析。

单一学科容易片面，需要整合多个视角。

7.3 批判性思维

关键原则：

- 质疑所有声称（包括AI的自我解释和这篇文章）
- 寻求可验证证据
- 区分相关性与因果性
- 警惕拟人化（AI“知道”、“理解”可能只是比喻）
- 保持认识论谦逊

---

8. 结语

通过“生成混乱中文句子”这个具体任务，我们观察到了当前AI的一些特征：

核心观察：

- AI主要基于概率分布优化，倾向生成高频模式
- 人类具备主动反向选择的能力，AI主要在新约束下重新优化
- 提示词工程是人类将创新意图转化为AI可执行约束的过程

- 创新能力可能涉及多个层次，AI较擅长优化层面
- AI的自我解释面临难以内部验证的困境

能力差异：

人类智能 = 优化 + 目标生成 + 框架质疑 + 价值判断 AI智能 = 主要是优化

这可能不是量的差距，而是结构上的差异。

实践意义：

理解这些差异有助于更有效地利用AI——将其作为能力辅助工具，发挥人机协作的优势。人类负责创新方向和框架设定，AI负责高效执行和大规模处理。

关于AI自身的解释，需要通过外部科学方法验证，而不应完全依赖其自我报告。保持批判性思维，对任何声称（包括这篇文章本身）保持审慎态度。

从自建Agent到Agent Runtime + Skill：AI系统架构的演进与收敛

>

一、问题本质：我们到底在构建什么？

在“大模型 + AI Agent”的语境中，一个核心问题逐渐浮现：

> AI 系统中的“智能”究竟由谁承担？

常见但不严谨的理解包括：“大模型是大脑”、“Agent 是类人智能体”、“有 LLM + Prompt 就足够”。这些说法在工程层面并不成立，需要还原为清晰的系统结构。

二、基础认知：从模型到系统

2.1 LLM：推理引擎，而非完整系统

大模型的本质是：

> 概率驱动的推理引擎（token → token）

其能力边界非常明确：能做输入 → 输出生成；不能做状态管理、多步骤执行、主动调用外部系统。最基础的系统形态是 `Prompt → LLM → Output`，因此：

> LLM 本身不构成一个可执行系统

2.2 Agent：任务编排与执行系统

在工程语境中，更准确的定义是：

> Agent 是一个 Orchestrator（编排器）

它的职责包括：理解任务、拆解步骤、选择工具（tool routing）、执行循环（execution loop）、管理状态（state）、根据结果持续调整策略。因此：

> Agent 不是“智能本体”，而是“控制系统”

2.3 Tool 与 Skill：能力的结构化表达

概念	含义
Tool	单一能力（API / 脚本）
Skill	能力封装（Prompt + Tool + 使用策略）

更精确地表达为：

> Skill = Tool + 使用方式 + 业务语义

Skill 的意义在于将能力结构化、将经验可执行化，使 Agent 可以直接调用，也是后续构建能力平台的基础单元。

三、两种系统构建方式

3.1 自建 Agent 架构

典型技术：LangChain、LangGraph。

```
User → Application → Custom Agent → LLM → Tools / APIs
```

开发者需要自行实现工具选择、执行循环、状态管理、工作流控制和错误处理。特点是控制力强、灵活性高，但开发成本和复杂度也相应更高。

3.2 Agent Runtime + Skill 架构

```
User → Application → Agent Runtime（SDK / Platform） → Skills / Tools
```

执行流程：理解任务 → 选择 Skill → 执行 → 分析结果 → 继续下一步。

应用侧只需提供任务（prompt / 参数）和上下文，并接收结果；编排能力由 Runtime 接管。

3.3 核心差异

维度	自建 Agent	Agent Runtime
编排逻辑	自己实现	内置
控制力	强	中 / 弱
开发成本	高	低
适用场景	复杂系统	标准流程

3.4 什么时候需要自建 Agent?

不需要自建的场景——流程线性、分支较少、以生成与调用为主，例如：

```
数据收集 → 信息分析 → 结构化输出 → 报告生成 → PPT 生成
```

结论：Agent Runtime + Skill 已足够。

必须自建的场景——多分支复杂逻辑、强状态依赖、高一致性与可靠性要求，例如自动交易系统、风控系统、多 Agent 协作系统。

结论：需要自建 Agent（如基于 LangGraph）。

四、Skill Loader 与能力体系

4.1 Skill Loader 的作用

Skill Loader 是连接 Agent 与能力体系的桥梁，作用包括动态加载 Skill、解耦能力实现、支持热插拔。其演进路径为：

```
自定义 Skill Loader → 兼容 MCP → 接入开放 Skill 平台
```

4.2 MCP + Skill + CLI 的三层结构

层级	作用
MCP（协议层）	标准化调用方式
Skill（能力层）	封装能力
CLI / API（执行层）	执行入口

本质目标：

> **构建一个“可被 Agent 调用的能力系统”**

五、Coding Agent：从专用工具到通用 Agent Runtime

从名称上看，“Coding Agent”似乎只是“用于写代码的 Agent”。但以 Claude Code 为代表的这类系统，其能力已明显外溢：除代码生成之外，还具备任务理解与拆解、多步骤执行、工具调用、结果迭代修正等能力——这些正是一个通用编排系统所需要的全部要素。

当进一步结合以下要素后：

- **MCP**（标准化工具协议）
- **Skill**（能力封装）
- **CLI / API**（执行入口）

其能力边界已经扩展为：

> **通用任务执行系统（General-purpose Agent Runtime）**

这类通用 Agent Runtime 通常以两种方式对外提供：

Agent SDK（可嵌入形态）：嵌入到业务系统中，应用侧控制调用方式，支持模型替换与 Skill 扩展。本质是开发者在使用 Agent 的同时仍然控制 Agent。

Agent Platform（平台形态）：Agent 运行在平台侧，编排逻辑由平台负责，开发者主要提供任务与能力，模型与执行逻辑可能绑定。本质是开发者使用的是一个“托管的 Agent 系统”。

两种形态的共同点是：**将 Agent Runtime 标准化、产品化，并对外提供通用能力**。区别仅在于控制权归属与组件可替换性。

六、架构收敛趋势

6.1 三层架构

```
┌──────────────┐
│ Agent Runtime │（可替换）
└──────────────┘
       │
       ↓
┌──────────────┐
│ Skill Layer  │（核心资产）
└──────────────┘
       │
       ↓
┌──────────────┐
│ Business Sys │
└──────────────┘
```

6.2 Agent 正在基础设施化

Agent Runtime 正在演变为标准化组件（SDK / 平台），开发者不再需要实现 Agent 内核，关注点转向 Skill 的构建与沉淀。

6.3 Skill 成为核心资产

Skill 的本质是**可执行的知识（Executable Knowledge）**：

传统知识	Skill
文档	可执行
人脑经验	可复用
零散信息	结构化能力

七、落地路径

核心思路只有一条主线：**先解耦，后平台化**。

第一步：用 Skill Loader 解耦

在 Agent SDK 尚未成熟稳定的当下，通过自定义 Skill Loader 将业务逻辑与 Agent 实现隔离。这不是长期方案，而是一种过渡手段——当 Agent SDK 成熟后，可以快速切换，不需要重写业务逻辑。

第二步：构建开放平台

当内部协议链路打通后，平台本身的能力扩展方式就变了：

> 以前是“写代码实现新功能”，现在是“写 Skill + 配置即上线”。

对内，业务团队直接组合 Skill 快速交付；对外，平台将 Skill 组合封装为能力接口，以 MCP 等协议提供给客户调用——客户看到的是“你能做什么”，而非内部如何实现。**能力上线即交付，不再有漫长的开发转化链路。**

八、结论

> “Coding Agent”不再只是“写代码的 Agent”，而是演化为一种通用 Agent Runtime。

> AI 系统的核心正在从“构建 Agent”，转向“沉淀 Skill”。

> Agent 是可替换的执行层，Skill 是长期复利的核心资产，Skill 平台是最终的交付加速器。

将执行路径固化为资产：一种面向长期运行的AI架构

> 关于 Skill – Script 分层结构的一种思考

在大模型被广泛用于自动化和 Agent 构建之后，一个结构性问题逐渐显现：系统往往仍然是“单次推理型”的。

每次任务执行，都重新规划步骤，重新生成操作路径。即便任务高度重复，成功经验也很少被固化为长期资产。执行依赖的是当下的推理能力，而不是历史上已经验证过的稳定结构。

这种形态在一次性任务中没有问题，但在需要长期运行的系统中，会逐渐暴露局限：相似任务反复消耗推理资源，执行路径存在波动，最优实践难以沉淀。

如果任务具有一定重复性，环境又相对稳定，那么仅依赖即时推理并不是一种长期结构。问题不在模型能力，而在系统形态。

方法与执行的分离

在人类实践中，方法与具体技能天然分层存在。方法相对稳定，变化缓慢；具体操作路径在实践中不断调整与优化。

当前很多大模型系统把“方法”和“执行”压缩在同一次推理中完成——策略、计划、动作生成全部发生在同一层。如果将两者分离，结构会更清晰。

可以抽象出三层：

- **方法层**：描述目标、约束与策略原则
- **执行层**：具体操作路径
- **评估层**：成功判定与退化检测

```
┌──────────────────────────────────┐
│              方法层              │
│   目标描述 · 约束条件 · 策略原则  │  ← 人为长期维护的核心
└──────────────────────────────────┘
                │ 指导 / 修复
                ▼
┌──────────────────────────────────┐
│              执行层              │
│     Script（可执行代码资产）      │  ← Python / JavaScript 等
└──────────────────────────────────┘
                │ 执行结果
                ▼
┌──────────────────────────────────┐
│              评估层              │
│       成功判定 · 退化检测         │  ← 触发修复或重生成
└──────────────────────────────────┘
```

这里所称的 Script，一般指**传统意义上的可执行代码**（例如 Python、JavaScript 等脚本或程序文件），而不是模型生成的临时文本。它是可以被版本化、运行、替换与优化的实际代码资产。

关键并不在分层本身，而在于执行层是否成为长期资产。

执行资产的形成与维护

在没有现成执行代码时，系统只能依赖方法层直接完成任务。这一阶段的目标不是效率，而是寻找稳定路径。

当某种操作方式在多次运行中表现出稳定成功，就具备被固化的条件。此时可以将成功轨迹抽象为一份传统代码形式的 Script，并作为默认执行方式。

从这一刻起，系统结构发生转变：执行不再依赖持续推理，Script 成为默认执行主体，推理只在必要时介入。当环境变化或执行退化时，再由方法层对 Script 进行修复或重生成，形成一个循环：

```
┌──────────────┐          ┌──────────────┐
│   Script     │──────────▶│  评估结果    │
│  默认执行    │          └──────┬───────┘
└──────────────┘                 │ 退化/失败
        ▲                        ▼
        │                 ┌──────────────┐
        └─────────────────│   方法层     │
   修复/重生成 │  条件介入  │              │
                          └──────────────┘
```

推理的角色从“持续操作者”转变为“条件性干预者”。

这一结构也改变了维护方式。在传统软件工程中，人为关注的核心是代码本身，优化、修复、重构都发生在代码层。在这种分层结构下，Script 由系统在方法层指导下生成与优化，人为长期关注的对象转向方法层——目标表达是否清晰，约束是否合理，策略是否稳健，优化原则是否正确。

代码从“持续人工雕刻的主体”转变为“在策略约束下可被替换与重生成的执行产物”。维护的重心上移到了方法与结构表达本身。

与常见 Agent 结构的差异

很多 Agent 系统强调循环推理：观察—思考—行动—再观察，依赖持续推理来保持适应性。

这里的结构不同之处在于：强调执行代码的固化，构建执行代码的生命周期，将推理限定为监督与修复机制。重点不是让系统“持续思考”，而是让系统“逐渐稳定”。

```
常见 Agent 结构        Skill-Script 结构
 ┌──────────────────────┐          ┌──────────────────────┐
 │  观察→思考→行动→…    │          │        方法层        │
```

```
│      持续推理驱动        │              │      仅在退化时介入        │
│                          │              └──────────────────────────┘
│   适应性强               │                          │
│   但难以沉淀             │                          ▼
└──────────────────────────┘              ┌──────────────────────────┐
                                          │   Script 默认执行        │
                                          │   稳定运行·可沉淀        │
                                          └──────────────────────────┘
                                                       │
                                                       ▼
                                          ┌──────────────────────────┐
                                          │         评估层           │
                                          └──────────────────────────┘
```

适用前提与边界

这种结构并非适用于所有场景。如果任务完全一次性，或环境高度随机且持续变化，或无法定义明确成功标准，那么执行路径难以稳定，固化也缺乏意义。

它更适合任务重复出现、环境局部稳定、成功可被明确判定的长期系统。

核心判断

可以将这一结构压缩为几个基本判断：

- 稳定成功路径应被外化为传统可执行代码资产
- 代码应承担默认执行职责
- 优化必须在评估约束下进行
- 人为维护的核心应位于方法层而非代码层

如果成立，系统就不再只是“每次重新解决问题”，而是在时间维度上逐渐形成稳定能力。它的改变不在于模型更强，而在于结构更长期。

大模型时代：中小公司如何寻找AI竞争空间

当前 AI 行业的格局已经逐渐清晰。大型科技公司正在集中资源投入两个核心方向：一是训练基础大模型，二是构建通用 AI Agent 平台。这两条路径都具有极高的进入门槛，对于大多数中小公司来说，很难在这些领域与大型科技公司正面竞争。

然而，这并不意味着 AI 时代只属于少数科技巨头。在应用层和行业层面仍然存在大量机会，关键在于找到与大公司不同的切入点。

大模型的两个天然限制

大模型和通用 Agent 虽然能力强大，但存在两个结构性的限制。

第一，私有数据的缺失。 大模型通过公开互联网数据训练，但企业内部数据、行业数据库、专业文献、内部报告等资源，往往因为商业价值或隐私属性而不会公开。即使是能力很强的大模型，在许多专业领域也无法直接获取最有价值的信息。

第二，行业工作流程的缺失。 许多行业的核心能力并不仅仅是知识本身，而是围绕知识形成的分析框架、判断逻辑与决策流程。法律分析、金融研究、医疗诊断、教育评估等领域，都有大量来自长期实践的隐性专业方法，很难通过通用模型直接获得。

这两个限制共同指向同一个结论：大模型擅长通用智能，但**行业数据与行业 workflow** 仍然是中小公司可以建立差异化能力的核心空间。

AI 软件生态的分层与中小公司的位置

从整体架构来看，AI 软件生态正在形成清晰的分层：

大模型 → 通用 Agent → 行业工具（Tools）

大型科技公司主要负责基础模型与平台层，中小公司可以在应用层和行业工具层建立自己的位置。随着 AI Agent 生态的发展，越来越多的平台通过工具调用（Tools）的方式扩展能力，这意味着行业专业能力只要封装清晰、接口标准，就能被各类 Agent 平台直接集成调用——中小公司并不需要构建完整的 AI 平台，同样可以成为生态中不可或缺的一部分。

围绕行业数据与行业 workflow，中小公司在应用层大致有三种产品形态可以切入。

三个可行的切入方向

第一，垂直领域 AI Agent —— 完整的行业 AI 系统。

这是最完整的产品形态，专注于某一特定行业或细分场景，例如法律助手、教育辅导、金融研究、医疗咨询等。系统需要整合行业数据、知识库与专业工作流程，独立面向终端用户提供服务。目标用户群体相对较小，但专业性更强，往往更容易形成稳定的商业模式。

第二，垂直 AI 功能模块 —— 行业专业能力的工具化封装。

许多行业中存在大量高频、专业、有方法论支撑的操作，例如合同条款提取、病历结构化解析、财务科目归类、试题难度评估、风险评分建模等。将这类能力封装为可调用的功能模块，不需要构建完整的产品体验，只需将专业操作逻辑结构化、接口标准化，即可通过 MCP、CLI + Skill 等方式被其他 AI Agent 或平台作为 Tools 集成调用。

需要注意的是，这类工具在使用时通常需要用户将私有数据输入进来处理，例如上传合同文本、病历内容、财务报表等，数据是“流入”工具的。对于数据安全敏感的大型企业来说存在一定顾虑，采购决策往往更为谨慎。因此这类产品面向中小企业或个人用户时推广阻力较小，进入大型企业市场则需要额外解决数据安全与合规问题。

第三，领域 AI Search —— 最轻量的垂直搜索工具。

专注于某一类专业数据的检索与整合，例如法律案例数据库、科研文献数据库、行业研究报告等。用户只需输入简短的查询意图或提问，即可轻松获取服务提供方基于垂直领域私有知识库检索整合后的专业结果或分析摘要，通常基于 RAG 架构实现，同样通过 MCP、CLI + Skill 等方式作为 Tools 被其他平台调用。

与垂直功能模块的关键区别在于数据流向：用户不需要输入任何私有数据，只需简单提问，私有知识库与分析能力完全在服务提供方一侧。这一特性使得领域搜索工具的使用门槛与传统搜索引擎相当，几乎不存在用户侧的数据泄露风险，更容易被各类规模的企业接受和采用。

后两种形态同属于被集成的专业能力模块，核心差异在于数据流向：**垂直功能模块需要用户将私有数据输入工具进行处理，数据安全门槛较高；领域搜索工具则将私有知识库与分析能力完全封存在服务方一侧，用户只需简单提问即可获取专业结果，不涉及任何私有数据的输入，数据风险极低，采购阻力也更小。** 选择哪种形态切入，取决于公司所掌握的核心资产是专业的处理能力、还是稀缺的行业数据资源。

结语

大模型时代的竞争，本质上并不仅仅是模型能力的竞争，而是**数据、知识与工作流程的竞争**。大型公司提供通用智能，中小公司则可以通过行业数据与专业经验构建差异化价值。

对于大多数中小公司来说，与其试图训练更大的模型，不如专注于一件更现实的事：**将行业知识与 workflow 工具化，并让这些工具能够被 AI 系统调用。** 在未来的 AI 生态中，这些专业工具很可能会像今天的软件插件或移动应用一样，成为整个系统中不可或缺的一部分。只要能够掌握某个领域的关键数据或方法论，即使是规模较小的公司，也仍然可以在 AI 时代建立自己的位置。

从Skill化到控制权：AI系统中的能力分层与资产保护

>

一、核心问题：能力分层与执行边界

在一些 AI 系统中，正在出现这样一种趋势：用 SDK + Runtime 提供执行环境，将任务拆解为 Skills，Runtime 负责编排，模型负责推理。这套模式提升了开发效率，但也引出一个更本质的问题：

> **哪些能力可以交给 Runtime 执行，哪些必须留在系统内部？**

这不是“用不用大模型”的问题，而是能力分层与执行边界的划分问题。

二、这个问题的前提

这里讨论的风险，有一个明确的前提：**你在使用外部系统**——无论是直接调用外部大模型、接入第三方 Agent，还是通过 SDK 对接外部服务，数据和上下文都需要离开你的系统边界。

如果你有能力自己部署本地模型，或者业务足够简单、本地轻量模型就能满足需求，数据根本不出去，Skill 化暂时可能也没有泄露风险需要考虑。

所以，分层不是普遍原则，而是**使用外部系统时的边界管理问题**。

三、进入外部系统意味着什么

使用外部系统时，所有传出去的内容本质上都一样：作为 prompt 的一部分明文传入模型参与推理。举几个典型的例子：

数据：用户数据、业务数据、RAG 检索内容，作为上下文直接传入，访问路径不再受控。

策略：prompt 模板、system prompt、参数配置，决定模型如何理解和执行任务，同样随请求传出。

结构：这类暴露更隐蔽。即使你没有把业务逻辑直接写进 prompt，外部也可以通过观察大量的输入输出反推出你的设计——调用顺序可以还原 workflow，结果分布可以逼近业务策略。

前两类是直接暴露，数据和策略作为 prompt 明文传出；第三类是间接暴露，通过观察大量输入输出来反推设计。两者的性质不同，但方向一致：**核心能力都存在被还原的可能**。

四、能力分层：三种典型情形

有效的应对方式不是加密，而是从源头控制哪些内容可以传出去，哪些不能。这需要对 Skill 做分层处理——以下不是严格的分类，更像是三种典型情形，用来说明“进不进外部系统”的判断逻辑。

通用能力不依赖私有数据，没有业务逻辑，文本总结、改写翻译、信息抽取都属于这类。这类 Skill 可以直接交给外部系统执行，暴露了也无妨。

从这里开始，判断逻辑就不再是“能不能暴露”，而是“暴露到什么程度”。

业务能力包含逻辑，但关键在于：结果对外暴露是可以接受的，不可接受的只是“怎么算出来的”。与其把数据传出去让外部处理，不如反过来——在你的系统内部封装为 API，让外部系统调用这个接口、接收返回结果。数据的计算和处理全程留在你的系统内部，对外部不可见。风控判断、推荐候选、用户标签这类都适用这个模式。

核心能力的情况更进一步：不只是逻辑不能暴露，连结果本身都会泄露设计。比如 RAG 的切分与检索方式，光看输出的召回内容，就能反推出你的索引结构和排序策略。这类能力唯一的保护方式，是让它根本不出现在外部系统的执行链上。

三类的本质区别只有一个维度：**暴露结果是否可以接受**。可以接受的，封装逻辑即可；不可以接受的，整个从外部执行链移走。

五、数据策略：最小输入，而非脱敏

许多系统选择对数据脱敏（替换姓名、隐藏手机号），但这只减少了识别风险，并没有减少信息暴露本身。更有效的策略是只提供完成任务所需的最小信息。

以“判断是否推荐某产品”为例：

❓ 直接传原始数据，暴露完整数据结构：

```
{
  "name": "张三",
  "age": 32,
  "income": 50000,
  "loan_history": [...],
  "credit_score": 0.82
}
```

❓ 只传决策所需特征，不暴露来源与结构：

```
{
  "risk_level": "high",
  "repayment_stability": "low"
}
```

本质上是从“给数据”变成“给结论所需的最小条件”。这一原则适用于所有需要向外部系统传递信息的场景。

六、设计原则：核心能力不可被还原

整篇收敛为一个判断框架：

能力类型	处理方式
通用能力	直接 Skill 化，交给外部系统
业务能力	工具化封装，接口调用
核心能力	内收，不进入外部执行链

最终目标不是“完全不暴露”，而是：

> **即使外部系统可以观测所有调用，也无法还原你的核心逻辑与数据结构。**

后记

写到这里，这本小册子的核心观点可以收敛为四条：

第一，AI 本质是「规律发现系统」，不是类人智能。它以统计和优化的方式从数据中提取规律，并不理解这些规律的含义，也不具备动机和意识。清醒认识这一点，是避免过度恐慌或过度神化的基础。

第二，人机协作的核心是人类承担策略层，AI 承担执行层。理解这一点，才能真正用好 AI——不是把它当成一个可以完全托管的系统，而是把它当成一个能力放大器。人的判断力、目标设定和价值选择，在可见的将来仍不能完全替代。

第三，AI 的创造力有真实边界。在 L1（约束下优化）层面，AI 已经非常强大；但在 L2-L4（目标生成、框架质疑、范式创新）层面，人类的参与仍然不可或缺。理解这个边界，有助于设定合理的预期。

第四，Skill 是 AI 时代最有价值的长期数字资产。可执行的、经过验证的方法论封装，正在成为 AI 应用生态中最有沉淀价值的部分。无论是个人还是公司，在 AI 时代积累的核心资产，首先是经过验证的 Skill 和工作流。

这四点未必正确，且可能认知过期，但它们是作者经过认真思考后形成的判断。希望它们对你有所启发，也欢迎质疑和讨论。

2026 年 3 月